

Exercises

Exercise 5: Visualising data using base R

Read Chapter 4 to help you complete the questions in this exercise.

1. As in previous exercises, either create a new R script or continue with your previous R script in your RStudio Project. Again, make sure you include any metadata you feel is appropriate (title, description of task, date of creation etc) and don't forget to comment out your metadata with a `#` at the beginning of the line.
2. You do not need to download anything for this exercise. You already have '*cardiacdata.txt*' in the **data** directory of your RStudio Project, from Exercise 3. If for some reason you haven't, go back to the **Data** link, download '*cardiacdata.xlsx*' and save it as a tab delimited file called '*cardiacdata.txt*' as you did before.
3. A quick reminder of what these data are. They come from a cohort study of the risk factors for cardiovascular disease, in which 163 adults aged between 55 and 75 were examined and a range of measurements taken: age and sex, systolic and diastolic blood pressure (mmHg), body mass index (kg m^{-2}), total cholesterol, HDL cholesterol and triglyceride (all mmol l^{-1}), units of alcohol drunk in the previous week, and smoking status. Note that you are importing the original file again, not the cleaned version you exported at the end of Exercise 4. Starting from the raw data every time, and doing your cleaning in a script, is what makes an analysis reproducible.
4. Import the '*cardiacdata.txt*' file into R using the `read.table()` function and assign it to a variable named `cardiac`. Use the `str()` function to display the structure of the dataset. As you found in Exercise 3, the `sex` and `smoking` variables are coded as integers, but they are really categories. Create a new variable in the `cardiac` dataframe for each of them, recoded as a factor with meaningful labels (`sex` is 1 for Female and 2 for Male; `smoking` is 1 for Current, 2 for Ex and 3 for Never). Use the `str()` function again to check the coding of these new variables. Almost every plotting function in this exercise treats a factor differently from a number, so this step is not optional housekeeping - get it wrong and your boxplots will come out as nonsense.
5. How many patients are there in each combination of smoking status and sex (hint: remember the `table()` function?)? Don't forget to use the factor recoded versions of these variables. Is the split between the smoking categories similar for women and men? Are any of the combinations small enough to worry about if you wanted to compare them?

6. The humble cleveland dotplot is a great way of identifying if you have potential outliers in continuous variables (see Section 4.2.4). Create dotplots (using the `dotchart()` function) for the following variables; `bmi`, `systolic`, `tchol` and `alcohol`. Do these variables contain any unusually large or small observations? Don't forget, if you prefer to create a single figure with all 4 plots you can always split your plotting device into 2 rows and 2 columns (see Section 4.4 of the book).

7. You met that extreme `bmi` value in Exercise 4, when you checked the ranges with `summary()` and found a maximum of 514.60. A body mass index of 514 is not possible: it is 51.46 with the decimal point in the wrong place. What the dotplot adds is not the discovery, but the damage. One bad value has made the plot useless for looking at the other 162 patients. Use the `which()` function to identify which observation it is, confirm the value with the square bracket `[ ]` notation, set it to `NA` as you did in Exercise 4, and then redraw the `bmi` dotchart. Now look again at all four plots from Q6. Several points still sit well away from the rest. Which ones, and what should you do about them?

8. Histograms are the standard way of looking at the distribution of a continuous variable (see Section 4.2.2). Create histograms for `bmi`, `systolic`, `tchol` and `alcohol`, either separately or in one figure as you did in Q6.

One thing to be careful of is that a histogram can look quite different depending on the number of 'breaks' used. The `hist()` function chooses breaks for you, but they are only a suggestion. Redraw the histogram of `systolic` twice, once with wide bins and once with narrow ones, and compare them. Which one would you show somebody, and what does the difference tell you about how much you should trust the shape of a histogram?

9. Scatterplots are great for visualising relationships between two continuous variables (Section 4.2.1). Before you draw one, stop and think about which variable belongs on which axis. The convention is that the explanatory variable goes on the x axis and the response goes on the y axis, so the choice is not cosmetic. Put them the wrong way round and you have quietly told your reader something you did not mean.

Start with `systolic` and `diastolic` blood pressure. Which would you put where, and why is that a harder question than it first looks? Now plot HDL cholesterol and triglyceride. Which of those two is the more sensible candidate for the x axis, and does the relationship you get make clinical sense to you?

Finally, look at `alcohol`. It is badly skewed, and skew is a nuisance in a plot because most of the points end up squashed into a corner. In Exercise 4 Q5 you took logs of `triglyceride` and it worked nicely. Try two transformations here, a square root with `sqrt()` and a natural log with `log()`, storing each as a new variable in the dataframe, then plot histograms of `alcohol` alongside both. One of the two goes badly wrong. Which one, and why? (hint: what is the smallest value of `alcohol`?)

10. When visualising differences in a continuous variable between levels of a factor (categorical variable) then a boxplot is your friend (avoid using bar plots - Google 'bar plots are evil' for more info). Create a boxplot to visualise the differences in total cholesterol at each level of smoking status (don't forget to use the recoded version of this variable you created in Q4). Include x and y axis labels in your plot. Make sure you understand the anatomy of a boxplot before moving on - please ask if you're not sure (also see Section 4.2.3 of the book).

An alternative to the boxplot is the violin plot, which combines a boxplot with a kernel density plot and shows you the shape of the distribution rather than just its quartiles. You will first need to install the `vioplot` package from CRAN and make it available with `library(vioplot)`. The `vioplot()` function then works in much the same way as `boxplot()`. Draw the same comparison as a violin plot and compare the two.

11. To explore the relationships between several continuous variables at once it's hard to beat a pairs plot. Create a pairs plot for the variables; `age`, `systolic`, `diastolic`, `tchol`, `hdlchol`, `triglyceride` and `bmi` (see Section 4.2.5 of the book for more details). If it looks a little cramped in RStudio then click on the 'zoom' button in the plot viewer to see a larger version.

One of the useful things about `pairs()` is that you can choose what goes into each panel. Modify your plot to add a smoother (a wiggly line) to the lower triangle, which makes the shape of each relationship much easier to see. The book and `?pairs` show you how. Then, just by looking, which pair of variables is most strongly related, and which variable is related to almost nothing?

11. (optional) `pairs()` will put whatever you like in the upper triangle, including the correlation coefficients themselves, but there is no ready made function for it so you have to write one. If you fancy a go, `?pairs` has an example you can adapt. Do the numbers agree with what you concluded by eye?

12. Every plot so far has appeared in the RStudio plot pane, which is fine while you are exploring but no use at all when you need a figure in a Word document, a PowerPoint slide or a report. Getting a plot out of R is a three step sandwich. You open a graphics device that is a file rather than a window, you draw the plot exactly as you have been doing, and then you close the device with `dev.off()`. Nothing appears on screen while you do it, and if you forget the `dev.off()` your file will be unreadable (see Section 4.5 of the book).

Redraw your boxplot from Q10 and save it twice into the `output` directory you created in Exercise 1, once as a pdf using `pdf()` and once as a png using `png()`. Open both files outside RStudio and try dropping the png into a Word document. Which format would you use for what, and why? (hint: enlarge each of them a long way and see what happens)

End of Exercise 5